

have multiple independent data spaces within the same physical memory.

When designing such systems, it is important to understand the maximum supported processors and external devices in a shared resource system. It also has implications on the maximum operating speed and other timing specifications for that interface. Signal integrity issues may also crop up due to long effective trace lengths and capacitive overloading of the source drivers.

Usually, a load of 30 pF is supported on each signal and thus the total load on each signal (device pin capacitance plus the board parasitic capacitance) should not exceed what each individual pin can support. If the system requires memory to be shared between multiple processors, choose processors that support on-chip bus arbitration so that no external glue logic is required. This will also provide better timing specs and hence support high-speed memory interfaces.

Application software

Existing code examples. Refer already available code examples. These are usually shipped out along with software development tools or are available as reference code in various application notes. The code is usually a tested code on evaluation platforms. This saves a lot of time and effort in debugging the code and getting it to work.

Available on-chip memory and support for off-chip memory. The size of the application and whether the code can fit into the memory of the chip or not should be determined. In addition to the on-chip memory, which is the fastest memory, designers should look at the possibility of using an external or off-chip memory such as SRAM, DDR-I or DDR-II.

Less frequently executed and less time-critical code could be run out of the off-chip memory. The off-chip memory could also be used to store a huge amount of raw data such as audio and video samples. Delay lines required for audio algorithms like

echo and reverb can make use of the off-chip memory.

MIPS computation and implications in real-time systems. This mainly depends on the software application and each software code has its own MIPS (million instructions per second) requirement. Assuming an audio decoder application where the decoded PCM stereo bitstream has to be driven out to a codec at the rate of 48 kHz, the time period between two stereo samples/word is $1/48 \text{ kHz} = 20.8 \mu\text{s}$. This means that the subsequent stereo sample should be ready within this time.

Normally, processing is carried out in frames. So instead of one stereo sample, you should consider a buffer. Assuming the output buffer size is 1024 stereo words, the application gets $1024 \times 20.8 \mu\text{s}$ to keep the next buffer of the same size ready.

Usually, the output buffers are ping-pong such that while the data from one buffer is driven out to the codec, the application running on the core fills up the other buffer. The decoder code running on the processor core should be executed to have the other buffer with newly decoded samples available within this specified frame rate. If the time to execute and keep the data ready in the buffer is equal to the output frame rate, it would mean that the code running on a processor is consuming all the available core MIPS.

Normally, the application code is optimised or a faster processor is chosen to ensure real-time data processing and avoid loss of meaningful audio or video data. The software programmer must carry out an MIPS estimate while selecting a processor. Development tools also support various ways to optimise the code for size or speed.

Software application running with OS (operating system) versus running without OS (standalone). Applications that are integrated with the OS need a different framework than those without the OS. For example, in the OS environment, there are unpredictable latencies involved with interrupt servicing. The interrupts are not immediately

served. The kernel controls how and when the threads are executed – basically time slicing. This has implications even on the way hardware is designed. Timings of certain signals used in an interface, if controlled inside the ISR, may not be met in the OS environment due to interrupt service latencies. This factor controls how a software/hardware is designed to meet the interface protocol requirements.

Know various software optimisation techniques. Development tools themselves can optimise the code for size (less memory) or speed (faster execution). Depending on the requirement, one of these optimisations can be achieved. Further optimisation can be achieved by hand-coding certain portions of the code and using lookup tables, built-in processor functions that efficiently make use of the on-chip hardware resources like circular buffers, on-chip hardware like accelerators, loop counters, on-chip MAC support, multiple execution units, multiple cores, and memory allocation between the core and direct memory access (DMA).

It is important to understand the internal memory organisation, interfacing of the DMA engine with the internal memory blocks and the internal bus architecture of a processor to add more optimisation to the code in terms of speed. Using multifunctional instructions reduces the code size as well as improves the execution speed as multiple operations are carried out simultaneously in a single instruction cycle.

Last but not the least!

Ensure that the device manufacturers provide good technical support through e-mails, field application engineers and online support forums. Good support will help you develop and debug IC- and system-level issues and therefore accelerate the time to market. ●

The author is a DSP applications project manager at Analog Devices India Private Limited